UNIT 1: PROBLEM SOLVING & COMPUTATIONAL THINKING

1.1 Problem Definition

Meaning of Problem Definition

Problem definition is the process of **clearly understanding and formally stating what needs to be solved**. A poorly defined problem almost always leads to an incorrect or inefficient solution, regardless of programming skill.

Understanding the Exact Requirement

- Read the problem statement carefully.
- Identify **what is expected**, not what appears obvious.
- Avoid assumptions that are not explicitly stated.
- Clarify ambiguous terms before proceeding.

Example:
If the problem states *"find the largest number"*, clarify:

- From how many numbers?
- Are numbers integers or real?
- Can numbers be negative?
- What if inputs are equal?

Identifying Key Elements

1. Inputs

- Data provided to the system.
- May come from user input, files, sensors, databases, or APIs.
- Type, range, and format of inputs must be clearly known.

Example:

- Integer array of size n
- String of maximum length 100

2. Outputs

- The result produced by the system.
- Output format must be precise.

Example:

- A single integer
- A sorted list
- A Boolean value (true/false)

3. Constraints

- Restrictions under which the solution must work.
- Influence algorithm choice and performance.

Common constraints:

- Time limits (e.g., execution within 1 second)
- Memory limits
- Input size limits
- Platform or hardware constraints

4. Assumptions

- Conditions assumed to be true if not specified.
- Must be reasonable and minimal.
- Should always be documented.

Example:

- Input will always be valid
- No division by zero unless specified

Importance of Clarity

- Prevents misinterpretation.
- Reduces rework.
- Ensures correctness and efficiency.
- Enables communication among team members and stakeholders.

1.2 Problem Identification

Real Problem vs Perceived Problem

Often, the initially observed problem is not the actual root problem.

- **Perceived Problem**: What appears on the surface.
- **Real Problem**: The underlying issue that must be solved.

Example:

- Perceived: “The application is slow.”
- Real: “Database queries are unoptimized.”

Correct problem identification ensures that effort is spent on the right solution.

Breaking Complex Problems into Sub-Problems

Large problems are difficult to solve directly. They should be decomposed into smaller, manageable parts.

Benefits:

- Simplifies thinking
- Enables parallel development
- Improves debugging and testing

Example:
Online shopping system:

- User authentication
- Product catalog
- Cart management
- Payment processing

Identifying Reusable Components

Reusable components are solutions or modules that can be applied in multiple contexts.

Examples:

- Sorting algorithms
- Input validation routines
- Authentication modules

Benefits:

- Reduces development time
- Improves reliability
- Encourages modular design

1.3 Problem Solving Approaches

Different problems require different approaches. No single method fits all problems.

1. Trial and Error

- Try possible solutions until the correct one is found.
- Simple but inefficient.
- Suitable only for small or non-critical problems.

Limitations:

- Time-consuming
- No guarantee of optimal solution

2. Divide and Conquer

- Break the problem into smaller sub-problems.
- Solve each independently.
- Combine solutions.

Examples:

- Binary search
- Merge sort
- Quick sort

Advantages:

- Reduces complexity
- Efficient for large datasets

3. Pattern Recognition

- Identify similarities or repeated structures in problems.
- Use known solutions to solve new problems.

Examples:

- Recognizing Fibonacci patterns
- Identifying repeated subproblems in dynamic programming

4. Abstraction

- Focus on essential details.
- Ignore irrelevant information.
- Create simplified models.

Example:

- Representing a real-world car as an object with attributes like speed, fuel, and direction.

5. Algorithmic Thinking

- Developing a clear, step-by-step solution.
- Ensures determinism and correctness.
- Essential for programming and automation.

1.4 Steps in Problem Solving

This is a structured lifecycle followed in software development and algorithm design.

Step 1: Define the Problem

- Clearly state the problem.

- Identify inputs, outputs, constraints, and assumptions.

Step 2: Analyze the Problem

- Understand relationships between inputs and outputs.
- Identify edge cases.
- Determine complexity requirements.

Step 3: Design Algorithm

- Create a logical sequence of steps.
- Can be represented using:
    - Pseudocode
    - Flowcharts
    - Decision tables

Step 4: Select Data Structures

- Choose appropriate data structures to store and process data.

Examples:

- Arrays for fixed-size data
- Linked lists for dynamic data
- Stacks and queues for ordered processing
- Trees and graphs for hierarchical relationships

Step 5: Implement Solution

- Convert algorithm into code.
- Follow coding standards.
- Ensure readability and maintainability.

Step 6: Test and Validate

- Verify correctness using test cases.
- Include:
    - Normal cases
    - Boundary cases
    - Invalid inputs

Step 7: Optimize

- Improve performance and resource usage.
- Reduce time and space complexity.

- Refactor code if necessary.

1.5 Computational Thinking

Computational thinking is a **problem-solving methodology inspired by computer science**, applicable even outside programming.

1. Decomposition

- Breaking a problem into smaller parts.
- Each part is easier to understand and solve.

Example:

- Processing payroll → attendance, salary calculation, deductions, reporting.

2. Pattern Matching (Pattern Recognition)

- Identifying similarities across problems.
- Enables reuse of solutions and algorithms.

Example:

- Searching problems using linear or binary search patterns.

3. Abstraction

- Removing unnecessary details.
- Creating general models that apply to many problems.

Example:

- Abstracting a bank account as balance, deposit, withdraw operations.

4. Algorithm Design

- Developing precise instructions to solve problems.
- Algorithms must be:
    - Correct
    - Finite
    - Unambiguous
    - Efficient

UNIT 2: ALGORITHMS & FUNDAMENTALS

2.1 Algorithm

Definition of an Algorithm

An **algorithm** is a **finite sequence of well-defined, unambiguous steps** that, when followed correctly, solves a specific problem and produces a result.

In simple terms:

An algorithm is a step-by-step method to solve a problem logically and systematically.

Example (informal):
To find the maximum of two numbers:

1. Read two numbers
2. Compare them
3. Display the larger number

Characteristics of an Algorithm

Every valid algorithm must satisfy **five essential characteristics**.

---

1. Input

- An algorithm may have **zero or more inputs**.
- Inputs are the data provided to the algorithm before or during execution.

Example:

- Numbers to be sorted
- Value of n for factorial calculation

Key point:

- Input values must be **clearly defined**.

---

2. Output

- An algorithm must produce **at least one output**.
- Output is the result of the algorithm's execution.

Example:

- Sorted list
- Factorial value

- Maximum number

Without output, an algorithm has no practical use.

---

3. Definiteness

- Every step of the algorithm must be **clear, precise, and unambiguous**.
- Instructions should not leave room for interpretation.

Incorrect:

- “Process the data properly”

Correct:

- “Add 1 to each element of the array”

---

4. Finiteness

- An algorithm must **terminate after a finite number of steps**.
- Infinite loops violate this condition.

Example:

- A loop that stops when a condition becomes false is finite.
- A loop without a termination condition is infinite and invalid.

---

5. Effectiveness

- Each step must be **simple and executable** in a finite amount of time.
- Steps must be realistic and practically implementable.

Example:

- “Add two numbers” is effective.
- “Sort infinitely large data instantly” is not effective.

2.2 Algorithm Representation

Algorithm representation is the way an algorithm is **expressed or documented** so that it can be easily understood and implemented.

---

1. Pseudocode

Pseudocode is a **language-independent, informal way** of writing algorithms using structured statements similar to programming languages.

Characteristics:

- No strict syntax rules
- Easy to read and understand
- Focuses on logic, not language

Example:

START

READ a, b

IF a > b THEN

  PRINT a

ELSE

  PRINT b

END IF

END

Advantages:

- Simple
- Ideal for exams and design
- Easy to convert into code

---

2. Flowcharts

A **flowchart** is a graphical representation of an algorithm using symbols connected by arrows to show control flow.

Common symbols:

- Oval → Start / End
- Parallelogram → Input / Output
- Rectangle → Processing
- Diamond → Decision
- Arrow → Flow of control

Advantages:

- Visual clarity
- Easy to understand for beginners
- Helps in tracing logic

Limitations:

- Difficult to draw for large algorithms
- Not space-efficient

---

3. Structured English

Structured English uses **simple English statements** arranged in a structured manner using keywords like IF, WHILE, FOR, etc.

Example:

- Read number
- If number is even
    - Display "Even"
- Else
    - Display "Odd"

Advantages:

- Easy for non-programmers
- Close to natural language
- Good for requirement documentation

2.3 Algorithm Constructs

Algorithm constructs define **how control flows** during execution. These are the building blocks of every algorithm.

---

1. Sequential Execution

- Statements are executed **one after another** in order.
- Default execution mode.

Example:

1. Read input
2. Perform calculation
3. Display output

No branching or repetition occurs.

---

2. Conditional Branching

- Allows decision-making.
- Execution path depends on a condition.

Types:

- IF
- IF–ELSE
- IF–ELSE IF

Example:

```
IF marks >= 40 THEN
  PRINT "Pass"
ELSE
  PRINT "Fail"
END IF
```

Used when different actions are required based on conditions.

---

3. Iteration (Loops)

- Repeats a block of statements until a condition is satisfied.
- Used when repetitive tasks are required.

Types:

- FOR loop (fixed number of iterations)
- WHILE loop (condition-based)
- DO–WHILE loop (executes at least once)

Example:

```
FOR i = 1 TO n
  PRINT i
END FOR
```

---

4. Recursion

- A function or algorithm calls itself to solve a smaller instance of the same problem.
- Must have:
    - Base case (termination condition)
    - Recursive case

Example:
Factorial:

factorial(n):

```
IF n == 0
  RETURN 1
ELSE
  RETURN n * factorial(n-1)
```

Advantages:

- Elegant and concise solutions
- Useful for problems like trees, graphs, divide-and-conquer

Disadvantages:

- Higher memory usage (stack)
- Can be inefficient if not carefully designed

2.4 Algorithm Analysis

Algorithm analysis evaluates **how efficient an algorithm is**, especially when input size increases.

Purpose of Algorithm Analysis

- Compare multiple algorithms
- Predict performance
- Choose the best algorithm for given constraints
- Optimize time and memory usage

Parameters of Algorithm Analysis

---

1. Time Complexity

- Measures **how execution time grows** with input size.
- Expressed using **Big-O notation**.

Common time complexities:

- O(1) – Constant time
- O(n) – Linear time
- $O(n^2)$ – Quadratic time
- O(log n) – Logarithmic time

Example:

- Loop running n times → O(n)
- Nested loops → $O(n^2)$

---

2. Space Complexity

- Measures **memory usage** of an algorithm.
- Includes:
    - Input space
    - Auxiliary space (extra memory)

Example:

- Algorithm using an extra array of size n → O(n) space
- Recursive algorithms use stack space

Trade-off Between Time and Space

- Faster algorithms often use more memory.
- Memory-efficient algorithms may take more time.
- Selection depends on constraints and requirements.

UNIT 3: COMPLEXITY ANALYSIS

3.1 Time Complexity

Meaning of Time Complexity

Time complexity represents **how the running time of an algorithm increases as the input size (n) increases**.

Important clarification:

- It does **not** measure actual clock time.
- It measures **number of basic operations** as a function of input size.

Best Case Time Complexity

- The **minimum time** required for algorithm execution.
- Occurs under **most favorable input conditions**.

Example:

- Linear search when the element is at the first position.
- Time complexity: **O(1)**

Usage:

- Rarely used for comparison.
- Gives optimistic view.

Average Case Time Complexity

- The **expected time** over all possible inputs.
- More realistic than best case.
- Often complex to calculate.

Example:

- Linear search where element can appear anywhere.
- Average time: **O(n)**

Usage:

- Useful when input distribution is known.

Worst Case Time Complexity

- The **maximum time** required under worst input conditions.

- Most commonly used in analysis and exams.

Example:

- Linear search when element is at last position or not present.
- Time complexity: **O(n)**

Reason for preference:

- Guarantees performance upper bound.
- Critical for real-time and large-scale systems.

3.2 Asymptotic Notations

Asymptotic notations describe **algorithm growth rate** as input size approaches infinity.

Big-O Notation – O(f(n))

Represents the upper bound (worst-case growth).

Meaning:

Algorithm will never take more than O(f(n)) time.

Example:

- A loop running n times → O(n)
- Nested loop → $O(n^2)$

Used for:

- Worst-case analysis
- Comparing algorithms

Omega Notation – Ω(f(n))

Represents the lower bound (best-case growth).

Meaning:

Algorithm will take at least Ω(f(n)) time.

Example:

- Linear search best case → Ω(1)

Usage:

- Minimum performance guarantee

Theta Notation – Θ(f(n))

Represents a tight bound (both upper and lower bounds).

Meaning:

Algorithm takes exactly f(n) time in all cases.

Example:

- Loop that always runs n times → Θ(n)

Preferred when:

- Algorithm behavior is consistent

Relationship Summary

Notation	Meaning	Case
O	Upper bound	Worst
Ω	Lower bound	Best
Θ	Tight bound	Exact

3.3 Common Time Complexities

Understanding growth rates is crucial for comparison.

O(1) – Constant Time

- Execution time does not depend on input size.

Examples:

- Accessing array element by index
- Simple assignment

Best possible complexity.

O(log n) – Logarithmic Time

- Input size reduces by a constant factor each step.

Examples:

- Binary search
- Tree traversal (balanced tree)

Highly efficient for large inputs.

O(n) – Linear Time

- Time increases linearly with input size.

Examples:

- Linear search
- Single loop over array

O(n log n) – Linearithmic Time

- Combination of linear and logarithmic behavior.

Examples:

- Merge sort
- Quick sort (average case)

Optimal for comparison-based sorting.

$O(n^2)$ – Quadratic Time

- Nested loops over same data.

Examples:

- Bubble sort
- Selection sort

Inefficient for large inputs.

$O(2^n)$ – Exponential Time

- Time doubles with each additional input.

Examples:

- Recursive Fibonacci
- Subset generation

Impractical for large inputs.

Growth Order (Fastest to Slowest)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

3.4 Complexity of Loops

Single Loop

```
for i = 1 to n
  print i
```

- Executes n times
- Time complexity: **O(n)**

Nested Loops

```
for i = 1 to n
  for j = 1 to n
    print i, j
```

- Inner loop runs n times for each outer loop
- Time complexity: **$O(n^2)$**

Dependent Loops

```
for i = 1 to n
  for j = 1 to i
    print j
```

- Total operations ≈ n(n+1)/2
- Time complexity: **$O(n^2)$**

Key exam point:

- Dependency does not reduce order, only constant factors.

Different Loop Ranges

```
for i = 1 to n
  for j = 1 to m
```

- Time complexity: **O(n × m)**

3.5 Complexity of Recursion

Recursive algorithms require **special analysis**.

Recurrence Relations

A recurrence relation expresses time complexity of a recursive function.

Example:

$T(n) = T(n-1) + O(1)$

- Linear recursion
- Time complexity: **O(n)**

Example:

$T(n) = 2T(n/2) + O(n)$

- Divide and conquer
- Time complexity: **O(n log n)**

Stack Depth

- Each recursive call occupies stack space.
- Stack depth = number of active recursive calls.

Example:

- Factorial recursion → depth = n
- Space complexity: **O(n)**

Function Call Overhead

Recursive calls involve:

- Parameter passing
- Return address storage
- Stack frame creation

Impact:

- Recursive solutions are often slower than iterative ones.
- Memory overhead increases.

Recursion vs Iteration (Exam-Oriented)

Aspect	Recursion	Iteration
Memory	Higher	Lower
Readability	High	Moderate
Speed	Slower	Faster
Stack usage	Yes	No

UNIT 4: ABSTRACT DATA TYPES (ADT)

4.1 ADT Concept

Meaning of Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** is a **logical or mathematical model of a data structure** that defines:

- **What data is stored**
- **What operations can be performed**
- **What behavior those operations exhibit**

It does **NOT** specify:

- How data is stored in memory
- Which programming language is used
- How operations are implemented internally

In short:

ADT focuses on *what* operations are performed, not *how* they are implemented.

Logical Description of Data

ADT describes data in terms of:

- Values
- Relationships between values
- Operations on those values

Example (Stack ADT):

- Data: Collection of elements
- Rule: Last-In-First-Out (LIFO)
- Operations: push, pop, peek

No mention of:

- Arrays or linked lists
- Memory allocation
- Pointers or indexes

Operations Without Implementation Details

ADT defines:

- Operation names

- Input/output behavior
- Preconditions and postconditions

Example:

- push(x) → inserts element x
- pop() → removes top element

ADT does **not** define:

- Whether stack uses array or linked list
- How resizing is handled

4.2 Benefits of ADT

ADT plays a critical role in **software engineering and data structure design**.

---

1. Modularity

- Each ADT represents an independent module.
- Implementation can be changed without affecting other parts.

Example:

- Stack implementation changes from array to linked list
- Code using stack remains unchanged

---

2. Reusability

- ADTs can be reused across multiple applications.
- Same ADT works in different contexts.

Example:

- Stack ADT used in:
    - Expression evaluation
    - Function calls
    - Undo/redo operations

---

3. Abstraction

- Hides internal complexity.
- User interacts only with defined operations.

Benefit:

- Reduces cognitive load
- Improves reliability

---

4. Maintainability

- Changes are localized to implementation.
- Fewer bugs during enhancement.

ADT-based systems are easier to:

- Debug
- Extend
- Refactor

4.3 Examples of ADT

Stack ADT

Concept:

- Linear ADT
- Follows **LIFO (Last In First Out)**

Operations:

- push()
- pop()
- peek()
- isEmpty()

Applications:

- Function calls
- Expression evaluation
- Backtracking

Queue ADT

Concept:

- Linear ADT
- Follows **FIFO (First In First Out)**

Operations:

- enqueue()
- dequeue()
- front()
- isEmpty()

Applications:

- Scheduling
- Buffer management
- BFS traversal

List ADT

Concept:

- Ordered collection of elements

Operations:

- insert()
- delete()
- search()
- traverse()

Implementation possibilities:

- Array list
- Linked list

Tree ADT

Concept:

- Non-linear hierarchical structure

Operations:

- insert()
- delete()
- traverse()
- search()

Applications:

- File systems
- Databases
- Expression trees

UNIT 5: ARRAYS

5.1 Definition of Array

An **array** is a **linear data structure** that:

- Stores elements of the **same data type**
- Uses **contiguous memory locations**
- Has a **fixed size**

Access is done using an **index**.

Contiguous Memory Allocation

- Elements are stored one after another in memory.
- Enables direct access using index.

Formula:

Address of A[i] = Base_Address + (i × size_of_element)

This property makes arrays extremely fast for access.

Fixed Size

- Size must be defined at creation.
- Cannot grow or shrink dynamically.

This leads to certain limitations (discussed later).

5.2 Types of Arrays

One-Dimensional Array

- Linear structure
- Single index

Example:

int arr[5];

Used for:

- Lists
- Simple data collections

Two-Dimensional Array

- Table or matrix structure
- Rows and columns

Example:

int matrix[3][3];

Used for:

- Matrices
- Grids
- Tabular data

Multi-Dimensional Array

- More than two dimensions

Example:

int arr[2][3][4];

Used for:

- Scientific computing
- Complex data modeling

5.3 Array Operations

Traversal

- Visiting each element exactly once
- Used for printing, processing

Time Complexity: O(n)

Insertion

Types:

- At end
- At beginning
- At specific position

Cost:

- Shifting elements required

Time Complexity:

- Best (end): O(1)
- Worst (beginning/middle): O(n)

Deletion

- Removing element
- Shifting required to fill gap

Time Complexity: O(n)

Searching

- Linear search → O(n)
- Binary search (sorted array) → O(log n)

Updating

- Changing value at a known index

Time Complexity: O(1)

5.4 Advantages of Arrays

Fast Access – O(1)

- Direct access using index
- No traversal required

This is the **biggest advantage** of arrays.

Easy Implementation

- Simple structure
- Easy to understand and use

Preferred when:

- Size is known in advance
- Fast access is required

5.5 Limitations of Arrays

Fixed Size

- Size cannot be changed at runtime
- Leads to either:
    - Overflow (if size is small)
    - Wastage (if size is large)

Memory Wastage

- Unused allocated space remains wasted
- Cannot return unused memory dynamically

Costly Insertion and Deletion

- Requires shifting elements
- Inefficient for frequent updates

Time Complexity: O(n)

ADT vs Array (Very Important for CCEE)

Aspect	**ADT**	**Array**
Nature	Logical	Physical
Focus	Operations	Storage
Implementation	Hidden	Explicit
Flexibility	High	Low
Example	Stack ADT	int arr[10]

UNIT 6: STACK

6.1 Stack Concept

A **Stack** is a **linear data structure** that follows the principle:

LIFO – Last In, First Out

Meaning:

- The element inserted **last** is removed **first**.
- All insertions and deletions happen from **one end only**, called the **Top**.

Real-world analogy:

- Stack of plates
- Browser back button
- Undo operations

6.2 Stack Operations

All stack operations are performed at the **Top**.

---

1. Push

- Inserts an element onto the stack.
- Increases the top pointer.

Overflow Condition:

- Occurs when stack is full and push is attempted.

Time Complexity: O(1)

---

2. Pop

- Removes the topmost element.
- Decreases the top pointer.

Underflow Condition:

- Occurs when stack is empty and pop is attempted.

Time Complexity: O(1)

---

3. Peek / Top

- Returns the top element **without removing it**.

Time Complexity: O(1)

---

4. IsEmpty

- Checks whether stack contains no elements.

Condition:

- top == -1

Time Complexity: O(1)

---

5. IsFull

- Checks whether stack has reached maximum capacity (array implementation).

Condition:

- top == size - 1

Time Complexity: O(1)

6.3 Stack Implementation

A. Stack Using Array

Characteristics:

- Fixed size
- Contiguous memory
- Faster access

Advantages:

- Simple implementation
- Cache friendly

Limitations:

- Stack overflow possible
- Fixed size causes memory wastage

B. Stack Using Linked List

Characteristics:

- Dynamic size

- Non-contiguous memory

Advantages:

- No overflow (until memory exhausts)
- Efficient memory usage

Limitations:

- Extra memory for pointers
- Slightly slower than array

Comparison (Exam-Oriented)

Feature	Array Stack	Linked List Stack
Size	Fixed	Dynamic
Overflow	Possible	Rare
Memory	Contiguous	Non-contiguous
Speed	Faster	Slightly slower

6.4 Applications of Stack

---

1. Function Calls

- Function calls stored in **call stack**.
- Stores:
    - Return address
    - Local variables
    - Parameters

---

2. Expression Evaluation

- Infix → Postfix / Prefix conversion
- Postfix expression evaluation

Stack helps manage **operator precedence**.

---

3. Parenthesis Checking

- Used to verify balanced brackets:

- (), {}, []

Algorithm:

- Push opening bracket
- Pop when closing bracket appears

---

4. Undo / Redo Operations

- Undo → Stack
- Redo → Another Stack

Used in:

- Text editors
- Image editors

---

5. Recursion Handling

- Each recursive call is stored in stack.
- Stack depth controls recursion limit.

UNIT 7: QUEUE

7.1 Queue Concept

A **Queue** is a **linear data structure** that follows:

FIFO – First In, First Out

Meaning:

- The element inserted first is removed first.
- Insertion at **Rear**
- Deletion from **Front**

Real-world analogy:

- Ticket queue
- Printer jobs
- CPU scheduling

7.2 Queue Operations

---

1. Enqueue

- Inserts an element at the **Rear** of the queue.

Overflow Condition:

- Queue is full.

Time Complexity: O(1)

---

2. Dequeue

- Removes an element from the **Front** of the queue.

Underflow Condition:

- Queue is empty.

Time Complexity: O(1)

---

3. Front

- Returns element at the front without removing it.

Time Complexity: O(1)

---

4. Rear

- Returns last inserted element.

Time Complexity: O(1)

7.3 Types of Queue

---

1. Simple Queue (Linear Queue)

Characteristics:

- Rear moves forward
- Memory wastage occurs after dequeue

Problem:

- Cannot reuse freed space

---

2. Circular Queue

Characteristics:

- Last position connects to first
- Rear wraps around

Advantage:

- Solves memory wastage

Condition:

- (rear + 1) % size == front → Full

---

3. Priority Queue

Characteristics:

- Each element has priority
- Higher priority served first

Types:

- Min priority queue
- Max priority queue

Used in:

- CPU scheduling

- Dijkstra algorithm

---

4. Deque (Double Ended Queue)

Characteristics:

- Insertion and deletion at both ends

Types:

- Input-restricted deque
- Output-restricted deque

7.4 Circular Queue (Important)

Why Circular Queue?

Problem in Simple Queue:

- After several dequeue operations, front moves forward.
- Space at beginning cannot be reused.

Circular Queue Solution:

- Rear moves to start after reaching end.
- Uses modulo arithmetic.

7.5 Applications of Queue

---

1. CPU Scheduling

- Processes scheduled in FIFO or priority order.

---

2. Printer Queue

- Print jobs handled sequentially.

---

3. BFS Traversal

- Breadth-First Search uses queue.
- Explores level by level.

STACK vs QUEUE (VERY IMPORTANT FOR CCEE)

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion	One end	Rear
Deletion	Same end	Front
Used in	Recursion	Scheduling

EXAM TRAPS TO REMEMBER

- Stack overflow vs queue overflow
- Circular queue avoids memory wastage
- Stack uses **one pointer (top)**, queue uses **two (front, rear)**
- Recursion = stack usage
- BFS = queue usage

UNIT 8: LINKED LIST

8.1 Concept of Linked List

A **Linked List** is a **linear dynamic data structure** in which:

- Elements (nodes) are stored in **non-contiguous memory**
- Each node contains:
    - **Data** (actual value)
    - **Link (pointer)** to the next node

Node Structure

| Data | Link |

Key points:

- Nodes are connected using pointers
- Memory is allocated **dynamically**
- Access is **sequential**, not random

Non-Contiguous Memory

- Nodes can be located anywhere in memory
- Links maintain logical order
- Eliminates memory wastage due to fixed size

8.2 Types of Linked List

---

1. Singly Linked List

Structure:

Data → Next → Data → Next → NULL

Characteristics:

- Each node points to the next node
- Traversal is **one-directional**

Advantages:

- Simple
- Less memory overhead

Limitations:

- Cannot traverse backward
- Deletion requires previous node reference

---

2. Doubly Linked List

Structure:

NULL ← Prev | Data | Next → Prev | Data | Next → NULL

Characteristics:

- Each node has two links:
    - Previous
    - Next

Advantages:

- Bidirectional traversal
- Easier deletion

Limitations:

- Extra memory for pointers
- Slightly complex operations

---

3. Circular Linked List

Structure:

- Last node points back to the first node
- No NULL pointer

Types:

- Circular singly linked list
- Circular doubly linked list

Advantages:

- Efficient traversal in loops
- Useful in scheduling problems

8.3 Linked List Operations

---

1. Insertion

a) At Beginning

- Create new node
- Point new node to head
- Update head

Time Complexity: O(1)

---

b) At End

- Traverse to last node
- Update last node's link

Time Complexity:

- Without tail pointer → O(n)
- With tail pointer → O(1)

---

c) At Specific Position

- Traverse to required position
- Adjust links

Time Complexity: O(n)

---

2. Deletion

- Remove a node and adjust links
- Requires traversal to locate node

Time Complexity: O(n)

---

3. Traversal

- Visit nodes one by one from head

Time Complexity: O(n)

---

4. Searching

- Sequential search

Time Complexity: O(n)

8.4 Array vs Linked List (IMPORTANT)

Parameter	Array	Linked List
Size	Fixed	Dynamic
Memory	Contiguous	Non-contiguous
Access	Random (O(1))	Sequential (O(n))
Insertion	Costly	Efficient
Deletion	Costly	Efficient
Memory Overhead	Low	High (pointers)

Exam Trap:

- **Arrays are faster for access**
- **Linked lists are better for frequent insertion/deletion**

UNIT 9: RECURSION

9.1 Definition

Recursion is a programming technique in which:

A function calls **itself** to solve a smaller instance of the same problem.

Used when:

- Problem can be divided into sub-problems of same type

9.2 Components of Recursion

Every recursive function MUST have:

---

1. Base Condition

- Stops recursion
- Prevents infinite calls

Example:

```
if (n == 0)
  return 1;
```

---

2. Recursive Call

- Function calls itself with reduced input

Example:

return n * factorial(n - 1);

9.3 Types of Recursion

---

1. Direct Recursion

- Function calls itself directly

Example:

```
fun() {
  fun();
}
```

---

2. Indirect Recursion

- Two or more functions call each other

Example:

A() → B()

B() → A()

---

3. Tail Recursion

- Recursive call is the **last statement**

Example:

```
tailFact(n, result) {
  if (n == 0)
    return result;
  return tailFact(n - 1, n * result);
}
```

Advantage:

- Can be optimized by compiler

9.4 Recursion Stack

Every recursive call creates an **activation record** on the stack.

Activation Record Contains:

- Function parameters
- Local variables
- Return address

Stack Depth

- Maximum number of active calls
- Directly affects memory usage

Example:

- Factorial of n → stack depth = n

9.5 Advantages of Recursion

---

1. Cleaner Code

- Shorter
- More readable

---

2. Natural Problem Representation

- Suitable for:
    - Tree traversal
    - Divide and conquer
    - Graph algorithms

9.6 Disadvantages of Recursion

---

1. Stack Overflow

- Occurs when recursion depth exceeds stack limit

---

2. Higher Memory Usage

- Each call consumes stack memory
- Less efficient than iteration

Recursion vs Iteration (VERY IMPORTANT)

Feature	Recursion	Iteration
Code	Cleaner	Longer
Memory	High	Low
Speed	Slower	Faster
Stack usage	Yes	No

COMMON CCEE EXAM TRAPS

- Missing base condition → infinite recursion
- Recursion always uses stack
- Tail recursion can be optimized
- Linked list traversal is always O(n)
- Linked list has memory overhead due to pointers

UNIT 10: TREES (CONFUSION-FREE EXPLANATION)

1. What is a Tree? (Big Picture)

A **tree** is a **non-linear, hierarchical data structure** used to represent **parent-child relationships**.

Think of:

- Family hierarchy
- Folder structure
- Organization chart

Key property:

- **No cycles**
- Exactly **one root**

10.1 Tree Terminology (FOUNDATION – MUST BE CLEAR)

Term	Meaning (Simple)
Root	Topmost node (starting point)
Node	Any element in tree
Edge	Connection between two nodes
Parent	Immediate predecessor
Child	Immediate successor
Leaf	Node with no children
Depth	Distance from root to node
Height	Longest path from node to leaf

🔴 **Common Confusion (EXAM TRAP)**

- **Depth** → counted from **root**
- **Height** → counted from **leaf**

10.2 Binary Tree (STRUCTURE ONLY)

Definition

A **Binary Tree** is a tree where:

Each node has **at most two children**
(left child and right child)

✓ No ordering rule
✓ Structure-based only

Important Point:

Binary Tree ≠ Binary Search Tree

10.3 Tree Traversals (HOW WE VISIT NODES)

Traversal = **order of visiting nodes**

1. Inorder (L → Root → R)

- Left subtree
- Root
- Right subtree

Used in:

- **BST** → gives **sorted order**

---

2. Preorder (Root → L → R)

- Root first

Used in:

- Copy tree
- Expression trees
- Prefix notation

---

3. Postorder (L → R → Root)

- Root visited last

Used in:

- Deleting tree
- Postfix expression evaluation

---

4. Level Order (BFS)

- Level by level
- Uses **Queue**

🔑 **One-Line Memory Trick**

Inorder → Sorted (BST)

Preorder → Root First

Postorder → Root Last

Level → BFS (Queue)

10.4 Binary Search Tree (BST) – WHERE CONFUSION STARTS

Definition (VERY IMPORTANT)

A **Binary Search Tree** is a **Binary Tree with ordering rule**:

Left Subtree < Root < Right Subtree

Key Difference

Binary Tree	**BST**
Structural	Ordered
No value rule	Left < Root < Right
Traversal order irrelevant	Inorder gives sorted output

10.5 BST Operations

1. Search

- Compare value with root
- Move left or right

Time Complexity

- Best/Average: O(log n)
- Worst (skewed): O(n)

---

2. Insert

- Insert like search
- Maintain BST property

---

3. Delete (MOST CONFUSING – EXAM FAVORITE)

Three cases:

1️⃣ **Leaf Node**
→ Simply delete

2️⃣ **Node with One Child**
→ Replace node with child

3️⃣ **Node with Two Children**
→ Replace with:

- Inorder Successor (smallest in right subtree)
  OR
- Inorder Predecessor (largest in left subtree)

10.6 Limitations of BST (WHY AVL EXISTS)

Problem: Skewed Tree

If data is inserted in **sorted order**, BST becomes:

```
1
 \
  2
   \
    3
     \
      4
```

Result:

- Height = n
- Search becomes **O(n)**

❗ BST loses efficiency

10.7 AVL Tree (CONFUSION KILLER SECTION)

What is AVL Tree?

An **AVL Tree** is:

A **self-balancing Binary Search Tree**

Meaning:

- Automatically maintains height balance

Balance Factor

Balance Factor = Height(Left) − Height(Right)

Allowed values:

−1, 0, +1

If balance factor goes outside this → **rotation needed**

AVL ROTATIONS (MOST CONFUSING PART – MADE SIMPLE)

WHY Rotations?

To **restore balance** after insertion/deletion

---

1. LL Rotation (Left-Left)

- Insertion in **left subtree of left child**
- Fix → **Right rotation**

---

2. RR Rotation (Right-Right)

- Insertion in **right subtree of right child**
- Fix → **Left rotation**

---

3. LR Rotation (Left-Right)

- Left child's right subtree
- Fix → **Left rotation + Right rotation**

---

4. RL Rotation (Right-Left)

- Right child's left subtree
- Fix → **Right rotation + Left rotation**

🔑 **Rotation Memory Trick (VERY IMPORTANT)**

First rotate the child, then rotate the root

TREE TYPE COMPARISON (CONFUSION DESTROYER)

Feature	Binary Tree	BST	AVL Tree
Children	≤ 2	≤ 2	≤ 2
Ordering	❌	✓	✓
Balance	❌	❌	✓
Search Time	O(n)	O(log n) avg	O(log n)
Worst Case	O(n)	O(n)	O(log n)

WHY B-TREE AND B+ TREE ARE NEEDED

Binary Search Trees (BST, AVL):

- Work well in **main memory (RAM)**
- Become inefficient when data is stored on **disk**

Disk Problem

- Disk access is **slow**
- BST/AVL causes **many disk reads** (height too large)

👉 **B-Tree and B+ Tree are designed to minimize disk access**
👉 Used in **databases and file systems**

B-TREE (Balanced Multi-way Search Tree)

Definition (EXAM READY)

A **B-Tree** is a **self-balancing multi-way search tree** in which:

- A node can have **more than two children**
- All leaves are at the **same level**
- Tree remains balanced automatically

Key Properties of B-Tree (Order = m)

Let **m = order of B-Tree**

1. Number of Children

- Max children = **m**
- Min children = **⌈m/2⌉** (except root)

---

2. Number of Keys in a Node

- Max keys = **m − 1**
- Min keys = **⌈m/2⌉ − 1**

---

3. Root Node

- Can have **minimum 2 children**
- Can have fewer keys than other nodes

---

4. Height

- Always **logarithmic**
- Guarantees **O(log n)** search, insert, delete

B-Tree Node Structure

| key1 | key2 | key3 |

| | | | |

C1 C2 C3 C4

- Keys act as **separators**
- Children hold ranges of values

Operations in B-Tree

Search

- Compare keys inside node
- Move to correct child

Time Complexity: O(log n)

Insert

- Insert at leaf
- If node overflows → **split**
- Middle key moves up

Delete

- Remove key
- If underflow → **merge or borrow**

Where B-Tree is Used

- File systems
- Database indexing
- Large storage systems

B+ TREE (MOST IMPORTANT FOR CCEE)

Definition (VERY IMPORTANT)

A **B+ Tree** is a **modified B-Tree** in which:

- **All actual data is stored only in leaf nodes**
- Internal nodes store **only keys (indexes)**
- Leaf nodes are **linked sequentially**

Key Characteristics of B+ Tree

1. Data Storage

- Internal nodes → **keys only**
- Leaf nodes → **keys + actual data**

👉 This is the **BIGGEST difference** from B-Tree

---

2. Leaf Node Linking

- Leaf nodes are connected using **linked list**

👉 Enables **fast range queries**

---

3. Height

- Same as B-Tree
- Always balanced

B+ Tree Node Structure

Internal Node

| k1 | k2 | k3 |

| | | | |

Leaf Node

| k1:data | k2:data | k3:data | → next leaf

Why B+ Tree is Preferred Over B-Tree

- Faster **range queries**
- More efficient disk access
- All data at same level → predictable performance

👉 **Databases almost always use B+ Tree**

B-TREE vs B+ TREE (VERY IMPORTANT TABLE)

Feature	B-Tree	B+ Tree
Data storage	Internal + leaf nodes	**Only leaf nodes**
Internal nodes	Keys + data	**Keys only**
Leaf nodes linked	❌ No	✓ Yes
Range queries	Slow	**Fast**
Disk efficiency	Good	**Better**
Used in databases	Rare	**Yes**

UNIT 11: SEARCHING ALGORITHMS

Searching = **finding a target element in a collection of data**

11.1 Linear Search

Concept

Linear search checks elements **one by one** from start to end until:

- Element is found OR
- Data ends

No assumptions about data order.

Working Steps

1. Start from first element
2. Compare with target
3. If match → stop
4. Else move to next element

Example

Array: [10, 25, 30, 45]
Search: 30

→ 10 ❌
→ 25 ❌
→ 30 ✓

Time Complexity

- **Best case:** O(1) (first element)
- **Average case:** O(n)
- **Worst case:** O(n)

Advantages

- Very simple
- Works on **unsorted data**
- No extra memory

Limitations

- Slow for large datasets

When to Use

- Small datasets
- Unsorted data
- One-time search

CCEE Trap

Binary search is NOT applicable on unsorted data

11.2 Binary Search

Concept

Binary search **repeatedly divides the search space into half.**

⚠ **Data must be sorted**

Divide and Conquer Approach

1. Find middle element
2. Compare target with middle
3. If smaller → search left half
4. If larger → search right half
5. Repeat

Example

Sorted Array: [10, 20, 30, 40, 50]
Search: 30

→ Middle = 30 ✓

Time Complexity

- **Best case:** O(1)
- **Worst case:** O(log n)

- **Average case:** O(log n)

Space Complexity

- Iterative → O(1)
- Recursive → O(log n)

Advantages

- Very fast for large data
- Efficient divide & conquer

Limitations

- Requires sorted data
- Not efficient on linked lists

CCEE Traps

✓ Binary search needs **random access**
✓ Not suitable for linked list
✓ Sorting cost ignored when asked *only search*

SEARCHING COMPARISON

Feature	Linear Search	Binary Search
Data order	Not required	Must be sorted
Time	O(n)	O(log n)
Data size	Small	Large
Structure	Any	Random access

UNIT 12: SORTING ALGORITHMS

Sorting = **arranging elements in a specific order** (ascending/descending)

12.1 Bubble Sort

Concept

- Repeatedly compares **adjacent elements**

- Swaps if in wrong order
- Largest element “bubbles” to end

Working

Multiple passes through array until sorted.

Time Complexity

- Best: O(n) (already sorted with optimization)
- Average: $O(n^2)$
- Worst: $O(n^2)$

Characteristics

✓ Simple
✓ Stable
❌ Very slow for large data

Use Case

- Educational
- Very small datasets

12.2 Selection Sort

Concept

- Repeatedly **selects minimum element**
- Places it at correct position

Working

1. Find minimum
2. Swap with first
3. Repeat for remaining

Time Complexity

- Best / Average / Worst → **$O(n^2)$**

Characteristics

✓ Simple
❌ Not stable
❌ Always slow

Use Case

- When swaps are costly but comparisons are cheap

12.3 Insertion Sort

Concept

- Builds sorted list **one element at a time**
- Similar to sorting playing cards

Working

- Take next element
- Insert into correct position in sorted part

Time Complexity

- Best: **O(n)** (already sorted)
- Average: $O(n^2)$
- Worst: $O(n^2)$

Characteristics

✓ Stable
✓ Efficient for **small or nearly sorted data**
✓ In-place

Use Case

- Small datasets
- Nearly sorted data

12.4 Merge Sort

Concept

- **Divide and conquer**
- Divide array into halves
- Sort each half
- Merge sorted halves

Time Complexity

- Best / Avg / Worst → **O(n log n)**

Space Complexity

- **O(n)** (extra memory)

Characteristics

✓ Stable
✓ Predictable performance
❌ Extra memory required

Use Case

- Large datasets
- External sorting
- When stability matters

12.5 Quick Sort

Concept

- Choose a **pivot**
- Partition array around pivot
- Recursively sort partitions

Time Complexity

- Best / Average → **O(n log n)**
- Worst → **O(n^2)** (bad pivot)

Space Complexity

- O(log n) average (recursion stack)

Characteristics

✓ Very fast in practice
✓ In-place
❌ Worst-case exists

CCEE Trap

Quick sort is **not stable** and **not guaranteed O(n log n)**

12.6 Heap Sort

Concept

- Uses **binary heap**
- Repeatedly removes max/min element

Time Complexity

- Best / Avg / Worst → **O(n log n)**

Space Complexity

- O(1) (in-place)

Characteristics

✓ Guaranteed performance
✓ No extra memory
❌ Not stable

SORTING COMPARISON (VERY IMPORTANT)

Algorithm	Best	Avg	Worst	Stable	Extra Space
Bubble	O(n)	$O(n^2)$	$O(n^2)$	✓	No
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	❌	No
Insertion	O(n)	$O(n^2)$	$O(n^2)$	✓	No
Merge	O(n log n)	O(n log n)	O(n log n)	✓	Yes
Quick	O(n log n)	O(n log n)	$O(n^2)$	❌	No
Heap	O(n log n)	O(n log n)	O(n log n)	❌	No

UNIT 13: HASHING

Hashing is one of the **fastest searching techniques** and is heavily tested conceptually in CCEE because it combines **data structures, algorithms, and complexity reasoning**.

13.1 Hashing Concept

What is Hashing?

Hashing is a technique used to:

Map a key directly to an index of an array (hash table) using a mathematical function.

Instead of searching sequentially or hierarchically, hashing tries to **directly compute the location** of data.

Key → Index Mapping

index = hash_function(key)

- **Key** → value to be stored or searched (e.g., roll number)
- **Hash function** → converts key into index
- **Hash table** → array where data is stored

Example

Hash table size = 10
Hash function: h(key) = key % 10

Key	Index
25	5
18	8
42	2

Why Hashing?

Searching Method	Time
Linear search	O(n)
Binary search	O(log n)
Hashing	**O(1) (average)**

👉 **Fastest searching technique (average case)**

13.2 Hash Functions

A **hash function** converts a key into a valid index of hash table.

Properties of a Good Hash Function

✓ Simple to compute
✓ Uniform distribution
✓ Minimizes collisions

---

1. Division Method

Formula

h(key) = key % table_size

Example

key = 123, table_size = 10

index = 123 % 10 = 3

Advantages

- Very simple
- Fast computation

Disadvantages

- Poor distribution if table size not chosen carefully

📌 **Exam Tip**: Table size should preferably be **prime**

---

2. Mid-Square Method

Steps

1. Square the key
2. Extract middle digits
3. Use them as index

Example

key = 123

$123^2 = 15129$

Middle digits = 12 → index

Advantages

- Uses all digits of key
- Better distribution than division

Disadvantages

- Slightly costly due to squaring

---

3. Folding Method

Steps

1. Divide key into equal parts
2. Add or fold parts
3. Result gives index

Example

Key = 123456

Split → 12 | 34 | 56

Sum → 102 → index

Advantages

- Useful for large keys
- Good for telephone numbers, IDs

13.3 Collision

What is a Collision?

A **collision** occurs when:

Two or more different keys map to the same hash index

h(k1) = h(k2)

Example

Hash function: h(key) = key % 10

Key	Index
25	5
35	5

👉 Collision occurs at index 5

Important Fact (CCEE FAVORITE)

❗ **Collision is unavoidable**
❗ Hashing ≠ collision-free
❗ Collision resolution is mandatory

13.4 Collision Resolution Techniques

When collision occurs, we must find **another place** to store data.

---

1. Linear Probing

Concept

- Check **next available index sequentially**

h(key), h(key)+1, h(key)+2, ...

Example

Index 5 occupied → try 6 → try 7

Advantages

✓ Simple
✓ Easy to implement

Disadvantages

❌ **Primary clustering**
(large blocks of occupied slots)

---

2. Quadratic Probing

Concept

- Probe indices using square increments

$h(key) + 1^2$, $h(key) + 2^2$, $h(key) + 3^2$, ...

Advantages

✓ Reduces primary clustering

Disadvantages

❌ Secondary clustering
❌ May not probe all slots

---

3. Double Hashing

Concept

- Uses **two hash functions**

h(key, i) = h1(key) + i × h2(key)

Advantages

✓ Best distribution
✓ Avoids both clustering types

Disadvantages

❌ More complex
❌ Extra computation

Collision Resolution Comparison

Method	Clustering	Performance
Linear	Primary	Poor
Quadratic	Secondary	Better
Double hashing	None	Best

13.5 Hash Table Operations

---

1. Insert

Steps:

1. Compute hash index
2. If empty → insert
3. If collision → apply probing

Time Complexity

- Average: **O(1)**
- Worst: **O(n)**

---

2. Search

Steps:

1. Compute index

2. If key not found → probe same sequence used during insertion

Important

Search must follow **same probing strategy**

---

3. Delete

Deletion is tricky because:

- Removing element may break probe chain

Solution:

- Use **lazy deletion** (mark as deleted)

Time Complexity Summary (VERY IMPORTANT)

Operation	Average	Worst
Insert	O(1)	O(n)
Search	O(1)	O(n)
Delete	O(1)	O(n)

UNIT 14: GRAPHS

Graphs are one of the **most powerful and widely used non-linear data structures**, used to model **networks, relationships, and paths**.

14.1 Graph Definition

What is a Graph?

A **graph** is a data structure defined as:

A **set of vertices (nodes)** and a **set of edges (connections)** between those vertices.

Formally:

G = (V, E)

Where:

- **V** = set of vertices
- **E** = set of edges

Examples of Graphs in Real Life

- Social networks (users → vertices, friendships → edges)
- Road networks (cities → vertices, roads → edges)
- Computer networks
- Dependency graphs

14.2 Types of Graphs

Understanding graph types is **very important for CCEE**, as many MCQs are classification-based.

---

1. Directed Graph (Digraph)

- Edges have **direction**
- Represented as ordered pairs (u → v)

Example:

A → B

Applications:

- Web links
- Task scheduling
- One-way roads

---

2. Undirected Graph

- Edges have **no direction**
- Connection is mutual

Example:

A — B

Applications:

- Friendship networks
- Undirected road networks

---

3. Weighted Graph

- Each edge has an associated **weight (cost, distance, time)**

Example:

A —(10)— B

Applications:

- Shortest path problems
- Network optimization

---

4. Unweighted Graph

- All edges have **equal weight**
- Only connectivity matters

Applications:

- BFS traversal
- Simple reachability problems

---

5. Cyclic Graph

- Contains **at least one cycle**
- A cycle is a path that starts and ends at the same vertex

Applications:

- Feedback systems
- Circular dependencies

---

6. Acyclic Graph

- Contains **no cycles**

Special case:

- **DAG (Directed Acyclic Graph)**

Applications:

- Scheduling
- Topological sorting

14.3 Graph Representation

Graphs must be stored in memory using proper representation.

---

1. Adjacency Matrix

Definition

A **2D matrix** of size V × V where:

- matrix[i][j] = 1 → edge exists
- matrix[i][j] = 0 → no edge

For weighted graph:

- Value = weight

Characteristics

Feature	Adjacency Matrix
Space	$O(V^2)$
Edge check	O(1)
Sparse graph	Inefficient
Dense graph	Efficient

Advantages

- Simple
- Fast edge lookup

Disadvantages

- High memory usage

---

2. Adjacency List

Definition

Each vertex stores a **list of its adjacent vertices**

Characteristics

Feature	Adjacency List
Space	O(V + E)
Edge check	O(deg(V))
Sparse graph	Efficient
Dense graph	Less efficient

Advantages

- Memory efficient
- Preferred in most applications

Representation Comparison (VERY IMPORTANT)

Parameter	Matrix	List
Space	$O(V^2)$	O(V+E)
Edge check	Fast	Slower
Traversal	Slower	Faster
Sparse graphs	Poor	Best

14.4 Graph Traversals

Traversal = **visiting all vertices systematically**

---

1. Breadth First Search (BFS)

Concept

- Visits vertices **level by level**
- Explores neighbors first

Uses

- **Queue**
- Works best for **unweighted shortest path**

Algorithm (High-Level)

1. Start from source
2. Enqueue source
3. Visit neighbors
4. Repeat

Time Complexity

- **O(V + E)**

Applications

- Shortest path (unweighted)
- Level order traversal
- Connectivity checking

---

2. Depth First Search (DFS)

Concept

- Goes **as deep as possible** before backtracking

Uses

- **Stack or Recursion**

Algorithm (High-Level)

1. Visit vertex
2. Recursively visit neighbors
3. Backtrack

Time Complexity

- **O(V + E)**

Applications

- Cycle detection
- Topological sorting
- Connected components

BFS vs DFS (VERY IMPORTANT)

Feature	BFS	DFS
Data structure	Queue	Stack / Recursion
Path	Shortest (unweighted)	Not guaranteed
Memory	Higher	Lower
Use case	Shortest path	Structure exploration

14.5 Shortest Path Algorithms

Used to find **minimum distance between vertices**.

---

1. Dijkstra's Algorithm

Concept

- Finds **shortest path from one source to all vertices**
- Works on **weighted graph**
- **No negative weights allowed**

Approach

- Greedy
- Repeatedly select nearest unvisited vertex

Time Complexity

- Using array: $O(V^2)$
- Using min-heap: $O((V + E) \log V)$

Applications

- GPS navigation
- Network routing

---

2. Floyd–Warshall Algorithm

Concept

- Finds **shortest paths between all pairs of vertices**
- Uses **dynamic programming**

Characteristics

- Works with negative weights
- Does **not** work with negative cycles

Time Complexity

- **$O(V^3)$**

Applications

- Dense graphs
- All-pairs shortest path problems

Dijkstra vs Floyd–Warshall (EXAM FAVORITE)

Feature	Dijkstra	Floyd–Warshall
Source	Single	All pairs
Negative edges	✗	✓
Technique	Greedy	Dynamic Programming
Complexity	Lower	Higher

14.6 Spanning Trees

What is a Spanning Tree?

A **spanning tree** is:

A subgraph that connects all vertices with **no cycles** and **minimum edges**

Properties:

- For V vertices → **V – 1 edges**
- Graph must be **connected**

Minimum Spanning Tree (MST)

An MST is a spanning tree with:

Minimum total edge weight

Used in:

- Network design
- Cable laying
- Cost optimization

MST Algorithms

---

1. Prim's Algorithm

Concept

- Greedy algorithm
- Starts from a vertex
- Grows tree by choosing minimum edge

Characteristics

- Works best on **dense graphs**
- Uses priority queue

Time Complexity

- $O(V^2)$ or $O(E \log V)$

---

2. Kruskal's Algorithm

Concept

- Greedy algorithm

- Sorts edges
- Adds smallest edge without forming cycle

Characteristics

- Uses **Disjoint Set (Union–Find)**
- Best for **sparse graphs**

Time Complexity

- O(E log E)

Prim vs Kruskal (VERY IMPORTANT)

Feature	Prim	Kruskal
Approach	Vertex-based	Edge-based
Graph type	Dense	Sparse
Data structure	Priority Queue	Union-Find
Cycle handling	Implicit	Explicit

UNIT 15: ALGORITHM DESIGN TECHNIQUES

(Think of this unit as "different ways to THINK while solving problems")

BIG PICTURE FIRST (MOST IMPORTANT)

All algorithm design techniques answer **one question**:

How should I approach solving this problem?

Technique	Core Idea
Divide & Conquer	Break → Solve → Combine
Greedy	Pick best local choice
Dynamic Programming	Remember past results
Brute Force	Try everything
Backtracking	Try → Fail → Undo
Branch & Bound	Prune bad choices
Stochastic	Use randomness

Keep this table in mind while reading further.

15.1 DIVIDE AND CONQUER

Core Idea

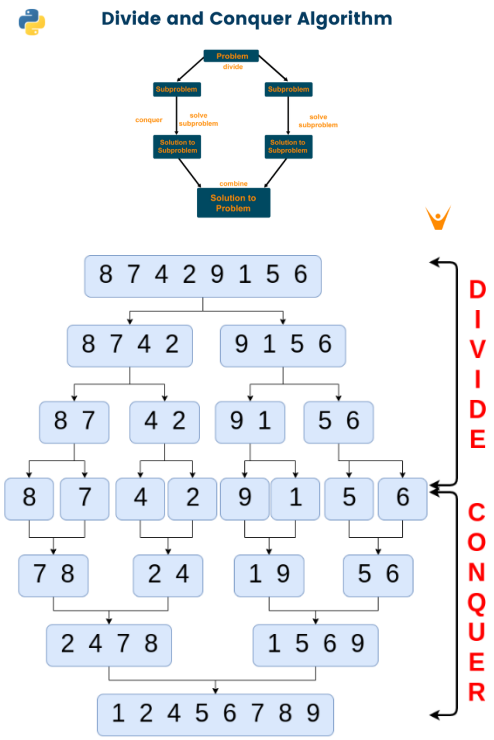
Divide the problem into smaller independent subproblems, solve them recursively, and combine their results.

Key Characteristics

- Subproblems are **independent**
- Recursion is heavily used
- Often improves time complexity

How Divide & Conquer Works

1. Divide problem into smaller parts
2. Solve each part recursively
3. Combine solutions



Merge Sort

Examples

Merge Sort

- Divide array into halves
- Sort each half
- Merge sorted halves
- **Time:** O(n log n)

Quick Sort

- Choose pivot
- Partition array
- Recursively sort parts
- **Average:** O(n log n)

Binary Search

- Divide search space by half
- **Time:** O(log n)

When to Use

✓ Problem can be broken into **independent parts**
✓ Recursive structure exists

Common Confusion

❌ Divide & Conquer ≠ Dynamic Programming

- D&C → **no reuse**
- DP → **reuse results**

15.2 GREEDY ALGORITHMS

Core Idea

At every step, choose the locally optimal option hoping it leads to a global optimum.

Key Characteristics

- Makes **immediate decisions**
- No reconsideration of past choices
- Very fast and simple

Examples

Dijkstra's Algorithm

- Always pick nearest unvisited vertex
- **Greedy choice:** shortest distance so far

Prim's Algorithm

- Grow MST by choosing smallest connecting edge

Kruskal's Algorithm

- Sort edges
- Pick smallest edge without cycle

When Greedy Works

✓ Problem has **greedy-choice property**
✓ Local optimum leads to global optimum

Common Confusion

❌ Greedy ≠ Dynamic Programming

- Greedy → **no revision**
- DP → **considers all possibilities**

15.3 DYNAMIC PROGRAMMING (MOST CONFUSING, MOST IMPORTANT)

Core Idea

Solve each subproblem once and store its result to avoid recomputation.

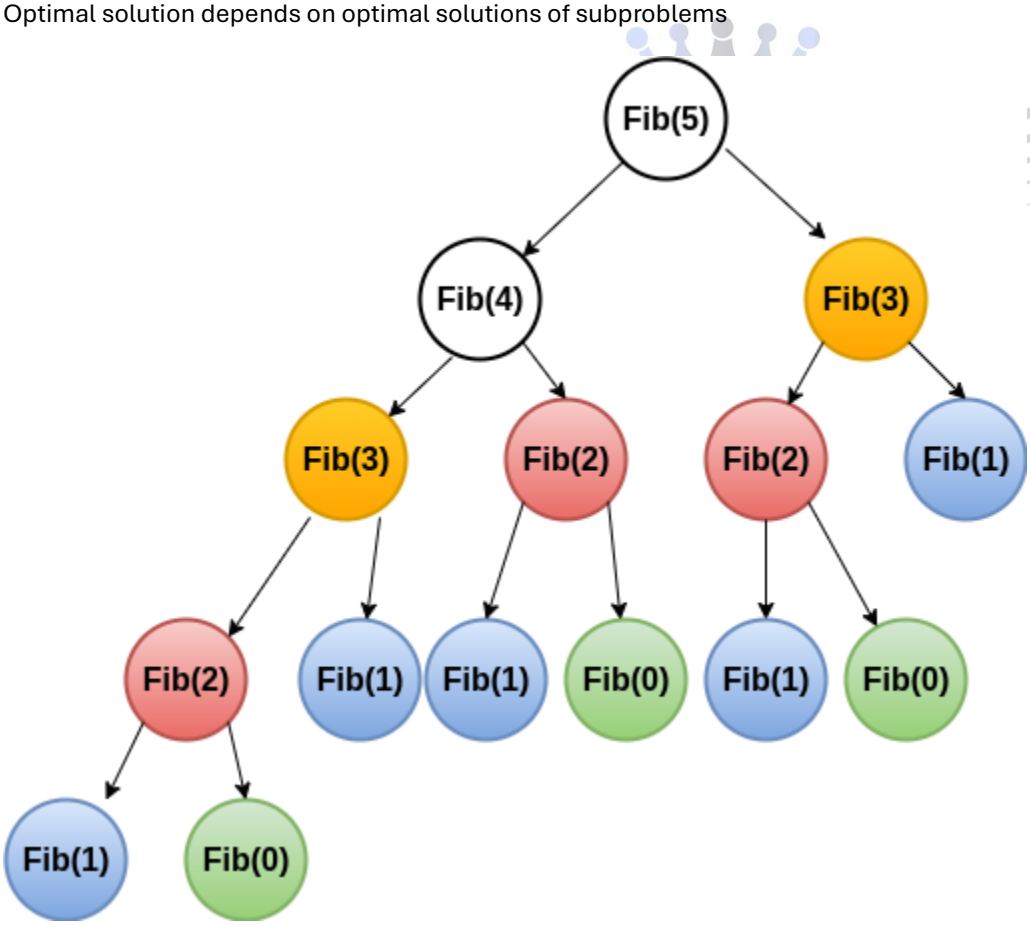
Two Mandatory Properties

1. Overlapping Subproblems

Same subproblem appears again and again

2. Optimal Substructure

Optimal solution depends on optimal solutions of subproblems



Example

- Fibonacci
- Knapsack
- Floyd–Warshall

DP Approaches

- **Top-Down (Memoization)** → recursion + cache
- **Bottom-Up (Tabulation)** → iterative table

DP vs Divide & Conquer (EXAM FAVORITE)

Aspect	Divide & Conquer	Dynamic Programming
Subproblems	Independent	Overlapping
Storage	No	Yes
Speed	Slower	Faster

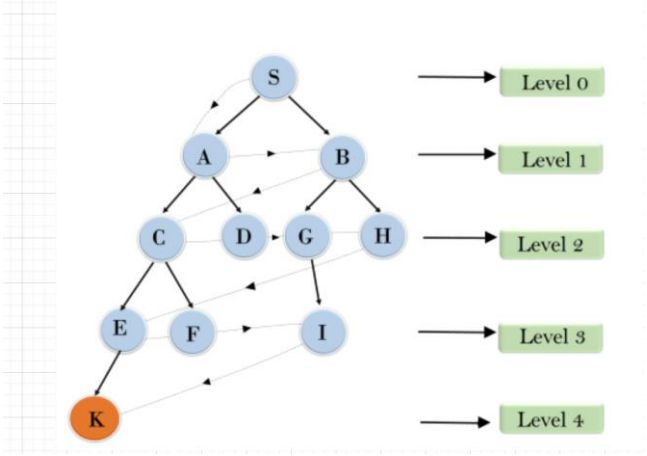
15.4 BRUTE FORCE

Core Idea

Try all possible solutions and select the correct or optimal one.

Characteristics

- Very simple
- No optimization
- Extremely slow for large inputs



Examples

- Linear search
- Trying all permutations
- Password cracking (conceptually)

When to Use

✓ Input size is **very small**
✓ Used as **baseline solution**

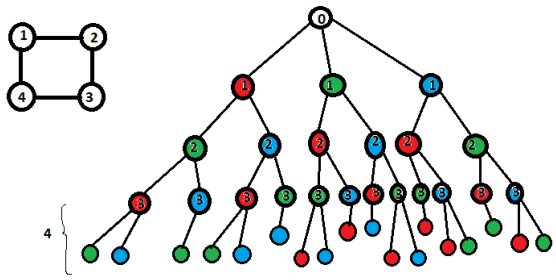
15.5 BACKTRACKING

Core Idea

Build solution step by step and abandon (backtrack) when it becomes invalid.

Key Difference from Brute Force

- Brute Force → tries everything
- Backtracking → **stops early when failure is detected**



Examples

N-Queen Problem

- Place queen
- Check safety

- Backtrack if conflict

Graph Coloring

- Assign color
- If conflict → undo

When to Use

✓ Constraint-based problems
✓ Decision tree structure

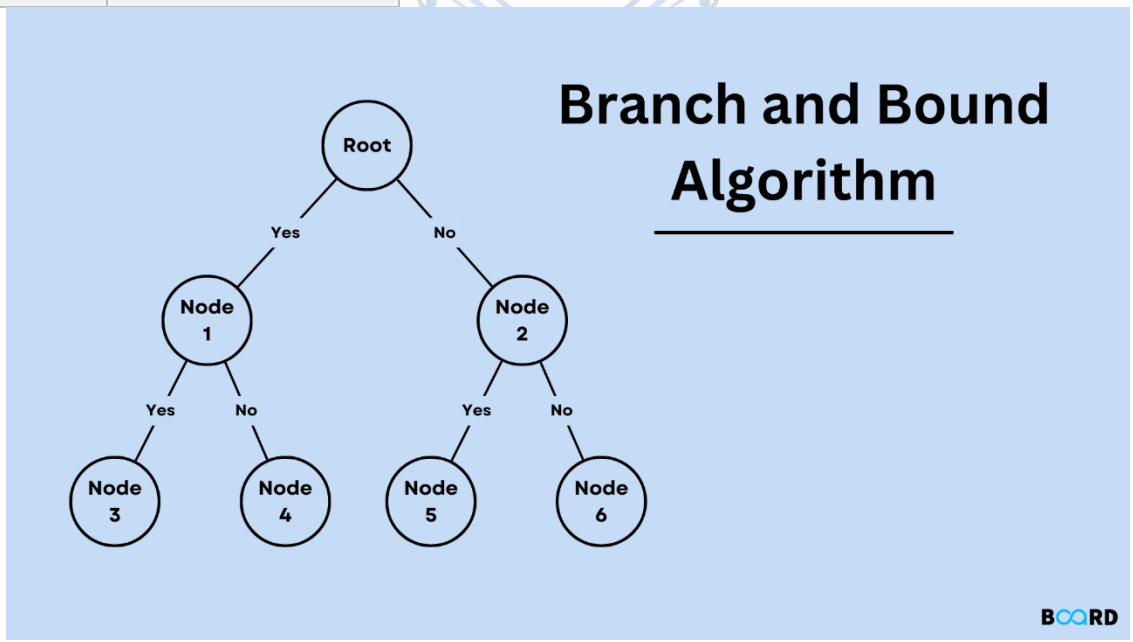
15.6 BRANCH AND BOUND

Core Idea

Like backtracking, but uses bounds to prune non-optimal branches early.

Key Difference from Backtracking

Backtracking	Branch & Bound
Decision problems	Optimization problems
Valid / invalid	Best / worst bound
Constraint based	Cost based



Examples

- Traveling Salesman Problem
- Job scheduling
- Knapsack (optimization version)

When to Use

✓ Optimization problems
✓ Need best solution, not just valid one

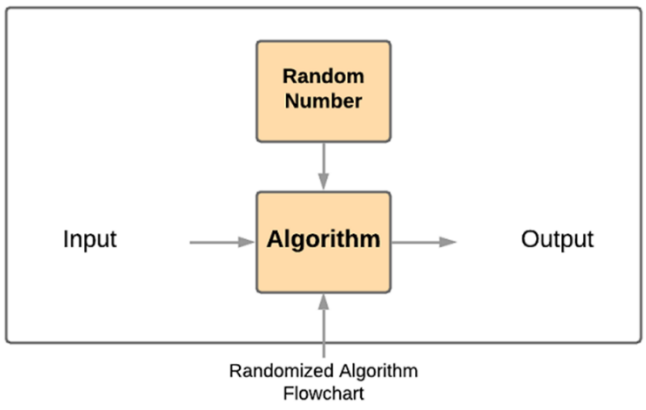
15.7 STOCHASTIC ALGORITHMS

Core Idea

Uses randomness to make decisions during execution.

Characteristics

- Non-deterministic
- Different output on different runs
- Useful when deterministic solutions are too slow



Randomized Algorithm Flowchart

Examples

- Randomized Quick Sort
- Monte Carlo algorithms
- Genetic algorithms

When to Use

✓ Very large or complex search space
✓ Approximate solution acceptable

MASTER CONFUSION-CLEARING TABLE (MOST IMPORTANT)

Technique	Decision Style	Revisits Choices?	Use Case
Divide & Conquer	Recursive split	✗	Sorting, searching
Greedy	Local best	✗	MST, shortest path
Dynamic Programming	Stored optimal	✓	Optimization
Brute Force	Try all	✗	Small input
Backtracking	Try + undo	✓	Constraint problems
Branch & Bound	Prune by cost	✓	Optimization
Stochastic	Random	Depends	Large search space